

Lecture 3: Linear Time Invariant Systems and Convolution

CSE 219: Signals and Linear Systems

Anik Saha

Adjunct Lecturer, Department of CSE, BUET

Systems and LTI Systems

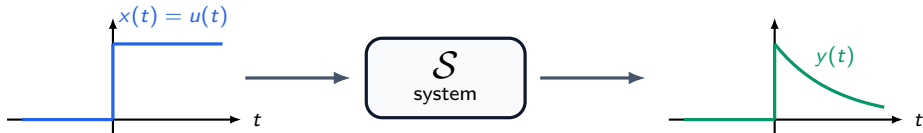
Signals go in. Signals come out.

A system transforms an input signal into an output signal

A system is something that takes a **signal** and produces another **signal**.

$$x(t) \longrightarrow \boxed{\text{system}} \longrightarrow y(t)$$

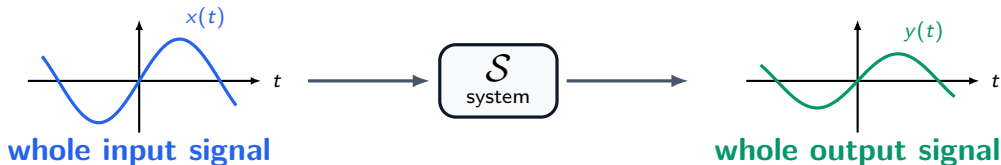
Continuous-time view



A system takes a whole signal, not just one number

Ordinary function: one number goes in, one number comes out

Signal system: one whole signal / function goes in, another whole signal / function comes out

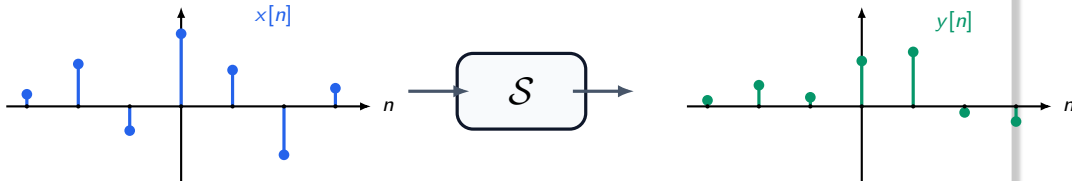


Discrete-time systems transform one sequence into another

In discrete time, the input is a whole sequence $x[n]$,
and the output is another sequence $y[n]$.

$$x[n] \longrightarrow \mathcal{S} \longrightarrow y[n]$$

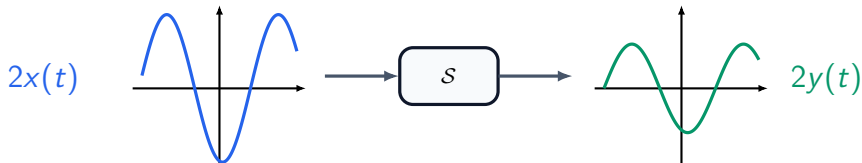
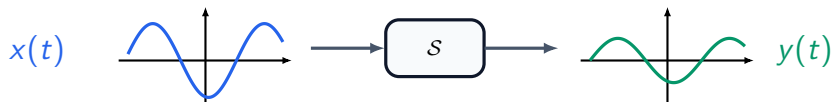
Example input-output pair



**Two Properties that will
make things easier**

Linear: scaling the input simply scales the output

If $x(t) \rightarrow y(t)$, then, $ax(t) \rightarrow ay(t)$.



Linear: adding inputs simply adds outputs

If $x_1(t) \longrightarrow y_1(t)$, and $x_2(t) \longrightarrow y_2(t)$, then, $x_1(t) + x_2(t) \longrightarrow y_1(t) + y_2(t)$.

Separate inputs

$$x_1(t) \longrightarrow y_1(t)$$

$$x_2(t) \longrightarrow y_2(t)$$

Treat the two pieces separately.

Input together

$$x_1(t) + x_2(t) \longrightarrow y_1(t) + y_2(t)$$

The output is just the sum
of the separate outputs.

Time-invariance

Input (Delayed)



Output (Delayed)



Input (Advanced)

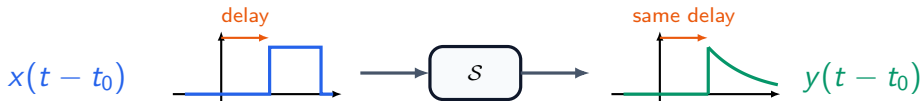


Output (Advanced)



Time-invariant: delaying the input only delays the output

If $x(t) \rightarrow y(t)$, then, $x(t - t_0) \rightarrow y(t - t_0)$.



An LTI system is linear and time-invariant

Linear

$$ax_1 + bx_2 \longrightarrow ay_1 + by_2$$

Time-invariant

$$x(t - t_0) \longrightarrow y(t - t_0)$$

Why we care: for LTI systems, one special response will eventually let us compute the output for **any possible input**.

A Detour through Convolution

Convolution is simply a mathematical operation that appears very
universally.

Convolution is just another operation on two sequences

$a[k]$	1	3	2	5	4
$b[k]$	2	-1	4	1	3
	$k = 0$	$k = 1$	$k = 2$	$k = 3$	$k = 4$

Add

$$(a + b)[k]$$

Add matching positions.

Subtract

$$(a - b)[k]$$

Subtract matching positions.

Convolve

$$(a * b)[k]$$

A different kind of mixing.

Convolution beautifully appears while multiplying polynomials

First polynomial

$$A(z) = 1 + 3z + 2z^2 + 5z^3 + 4z^4$$

1	3	2	5	4
a_0	a_1	a_2	a_3	a_4

Second polynomial

$$B(z) = 2 - z + 4z^2 + z^3 + 3z^4$$

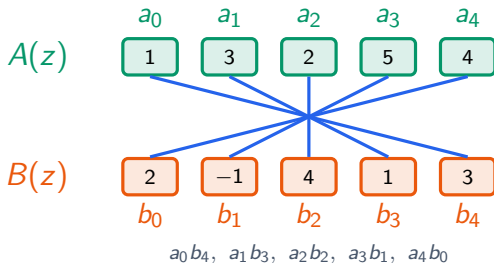
2	-1	4	1	3
b_0	b_1	b_2	b_3	b_4

Question: what is the coefficient of z^4 in $A(z)B(z)$?

The coefficient of z^4 comes from criss-crossed pairings

To obtain z^4 , we collect all products whose powers add to 4.

Pair coefficients whose indices add to 4



$$c[4] = 1 \cdot 3 + 3 \cdot 1 + 2 \cdot 4 + 5(-1) + 4 \cdot 2 = 17.$$

The crossed-pairing rule has a compact formula

If

$$A(z) = \sum_k a[k]z^k, \quad B(z) = \sum_k b[k]z^k,$$

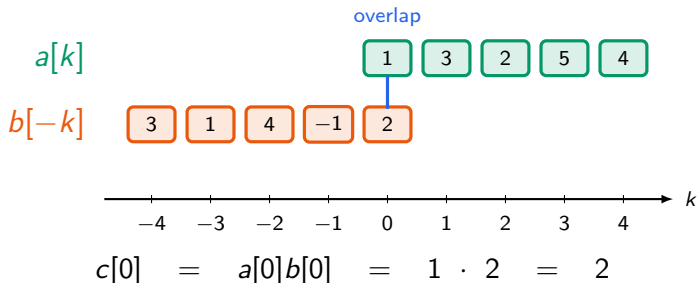
then the coefficient of z^n in $A(z)B(z)$ is

$$c[n] = \sum_k a[k] b[n - k].$$

Reflection turns crossed pairings into vertical pairings

The formula $b[n - k]$ means: reflect one sequence, then shift it.

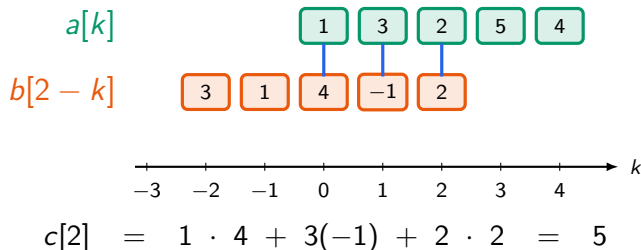
Start with $n = 0$: compare $a[k]$ with $b[-k]$



Sliding the reflected sequence gives the next coefficients

For $c[n]$, we compare $a[k]$ with $b[n - k]$.
The shift n decides which products line up vertically.

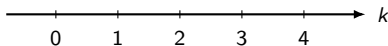
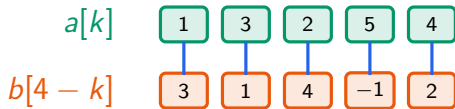
Now take for example $n = 2$: compare $a[k]$ with $b[2 - k]$



At $n = 4$, the vertical products give the z^4 coefficient

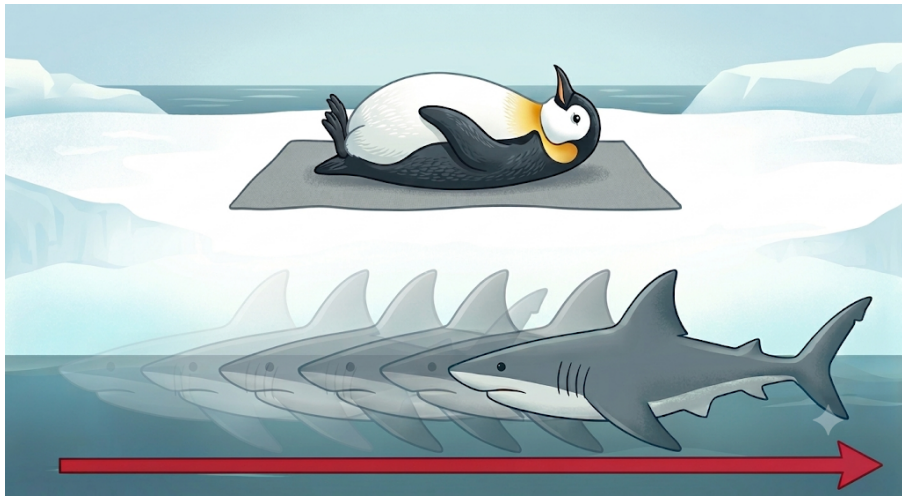
Now compare $a[k]$ with $b[4 - k]$. All five products line up.

The same $c[4]$ as before, now as vertical multiply-and-add



$$c[4] = 1 \cdot 3 + 3 \cdot 1 + 2 \cdot 4 + 5(-1) + 4 \cdot 2 = 17$$

Convolution: Keep the first signal just as is, reflect the second, slide rightward, find overlaps, then multiply-and-add!



This sliding multiply-and-add operation is convolution

$$c[n] = (a * b)[n] = \sum_{k=-\infty}^{\infty} a[k] b[n - k]$$

1. Reflect

$$b[k] \longrightarrow b[-k]$$

Turn one sequence around (time-reversal).

2. Shift

$$b[-k] \longrightarrow b[n - k]$$

Right shift the reflected sequence to the position n .

3. Multiply, add

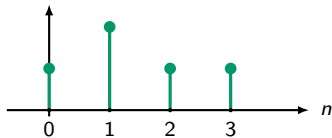
$$\sum_k a[k] b[n - k]$$

Sum of vertical products give one output value $c[n]$.

Practice: compute the convolution of two short sequences

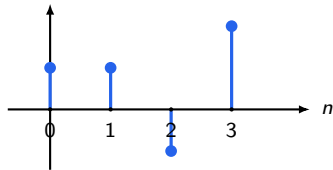
$x[n]$

$$x[0] = 1, x[1] = 2, x[2] = 1, x[3] = 1$$



$h[n]$

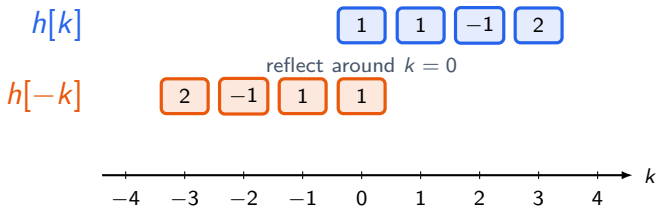
$$h[0] = 1, h[1] = 1, h[2] = -1, h[3] = 2$$



First reflect one sequence before sliding

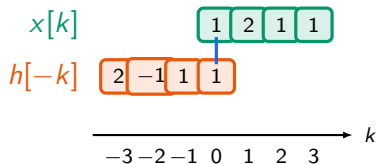
For convolution, we compare $x[k]$ with $h[n - k]$.
So the second sequence is first reflected: $h[k] \rightarrow h[-k]$.

Reflection of $h[k]$



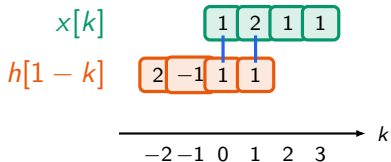
Slide the reflected sequence: first two output samples

$n = 0$: compare $x[k]$ with $h[-k]$



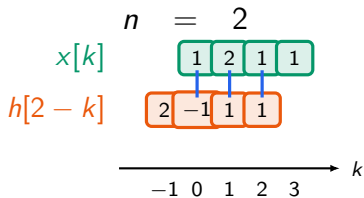
$$y[0] = 1 \cdot 1 = 1$$

$n = 1$: shift right by one



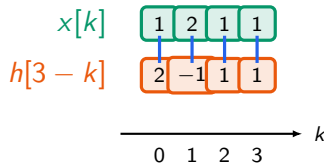
$$y[1] = 1 \cdot 1 + 2 \cdot 1 = 3$$

The middle positions have the largest overlap



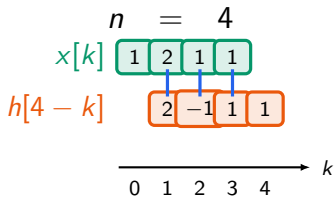
$$y[2] = 1(-1) + 2(1) + 1(1) = 2$$

$n = 3$: all four samples overlap

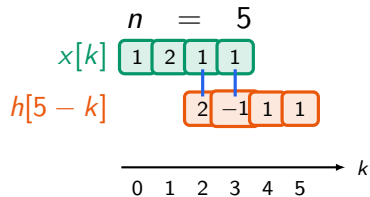


$$y[3] = 1(2) + 2(-1) + 1(1) + 1(1) = 2$$

As we keep sliding, the overlap becomes smaller

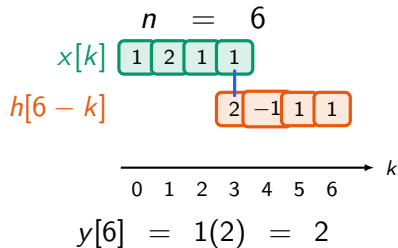


$$y[4] = 2(2) + 1(-1) + 1(1) = 4$$



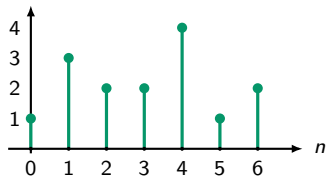
$$y[5] = 1(2) + 1(-1) = 1$$

The last overlap gives the final sample



Final output

$$y[n] = \{1, 3, 2, 2, 4, 1, 2\}$$



Think: can we shuffle the order?

We have defined convolution as

$$(a * b)[n] = \sum_k a[k] b[n - k].$$

But what if we swap the two sequences?

Does the order of convolution matter?

Answer: Sure! Convolution does not care about the order

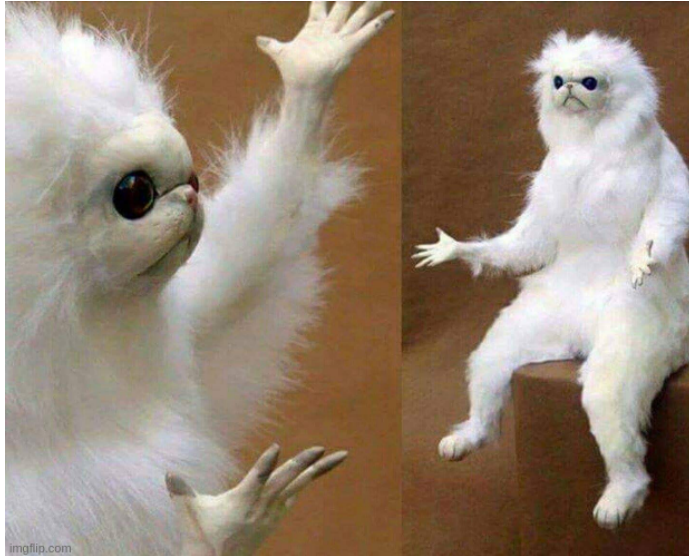
$$a * b = b * a$$

Polynomial analogy

$$A(z)B(z) = B(z)A(z)$$

Since convolution appeared from polynomial multiplication, swapping the two polynomials should not change the final coefficients.

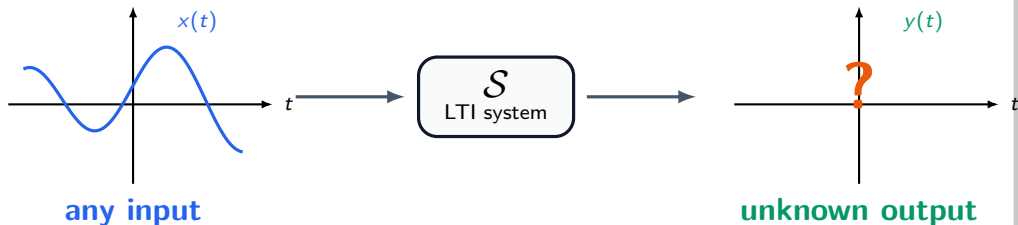
But why are we doing all these...?



**Why Convolution Matters
(so very much)!**

Our goal: find the output of LTI System for any input signal

The ambitious problem



Can we find a systematic method that works for any possible $x(t)$?

Think: how do we solve a difficult problem for any input?

Before signals, let us look at a familiar number problem.

Question

How many positive factors does 72 have?

Of course, we could check $1, 2, 3, \dots, 72$
one by one. But there is a better idea.

The easy pieces are prime powers

$$72 = 2^3 \cdot 3^2$$

Factors from 2^3

$$2^0, 2^1, 2^2, 2^3$$

$$3 + 1 = 4$$

There are 4 choices
for the power of 2.

Factors from 3^2

$$3^0, 3^1, 3^2$$

$$2 + 1 = 3$$

There are 3 choices
for the power of 3.

Then we combine the small answers

Each factor is made by choosing one power of 2 and one power of 3.

Number of choices

$$(3 + 1)(2 + 1) = 4 \cdot 3 = 12$$

So 72 has 12 positive factors.

The philosophy: solve simple pieces, then combine

Step 1

Solve for the simplest inputs.

Step 2

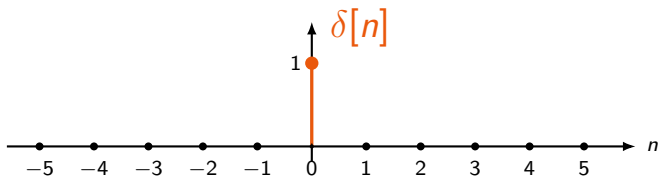
Break any input into simple pieces.

Step 3

Combine the small answers gracefully.

The simplest input signal is the impulse function

The discrete-time impulse



It is zero everywhere except at one instant.

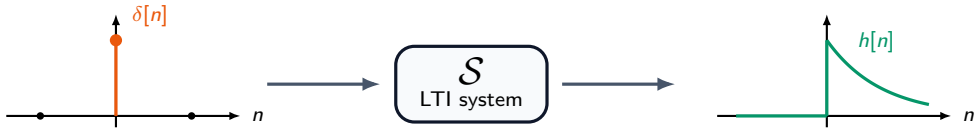
The impulse response: poking a sleeping cat...



The impulse response is the system's signature

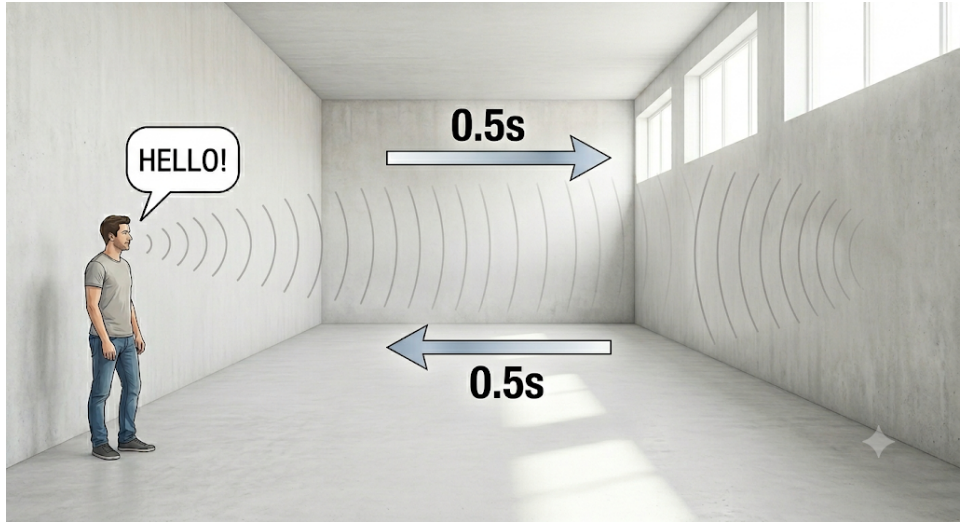
Given one impulse, the output we observe is called the **impulse response**.

Definition



$$\delta[n] \longrightarrow h[n]$$

An example: one *hello* can create many echoes

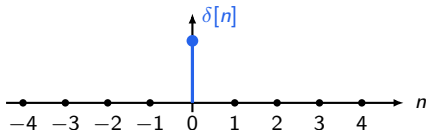


In this toy echo system, the impulse response is a step

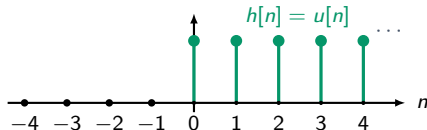
One short input sound produces repeated equal echoes:

$$\delta[n] \longrightarrow h[n] = u[n].$$

Input: one short *hello*



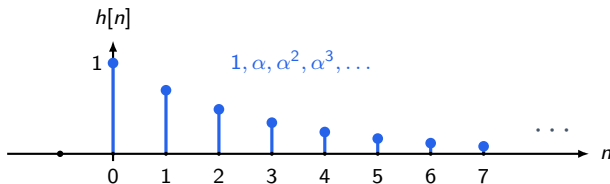
Output: repeated echoes



Think: what does this new impulse response mean?

$$\delta[n] \longrightarrow h[n] = \alpha^n u[n], \quad 0 < \alpha < 1.$$

Impulse response

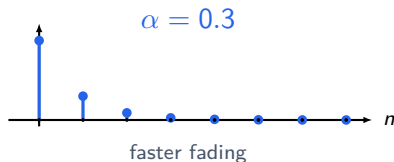


What does it mean physically? What if we change α ?

Answer: α controls how quickly the echoes fade

α is the fraction of echo strength that survives after each reflection.

Larger α means slower fading



Large α : echoes stay longer.

Small α : echoes die quickly.

Think: what impulse response gives only a delay?

Suppose a system does nothing except delay the input by 3 samples:

$$y[n] = x[n - 3].$$

What should its impulse response be?

Answer: a pure delay has a shifted impulse response

Impulse in, delayed impulse out

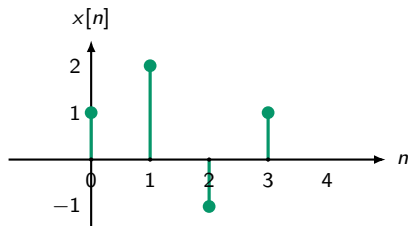


$$h[n] = \delta[n - 3]$$

Continuous-time version: $h(t) = \delta(t - 3)$.

Now let's send a four-sample input to our toy echo system!

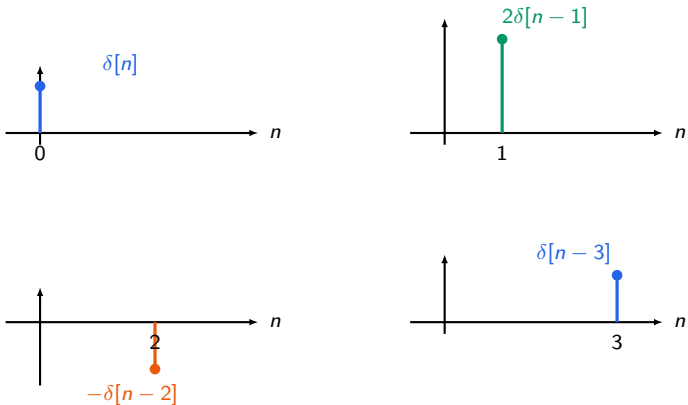
$$x[n] = \delta[n] + 2\delta[n - 1] - \delta[n - 2] + \delta[n - 3].$$



Can we find the output using only the response to one impulse?

Let's first break the input into scaled and shifted impulses!

Four simple pieces



LTI tells us the output of each shifted impulse...

$$\delta[n] \longrightarrow h[n] = u[n].$$

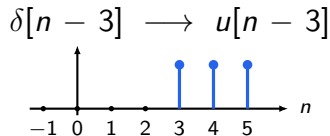
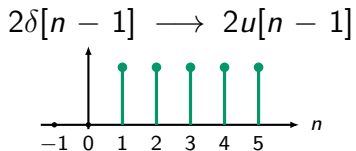
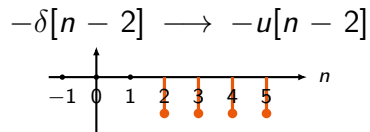
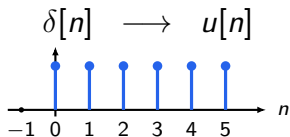
Linearity and Time invariance simplifies everything beautifully!

$$\delta[n] \longrightarrow h[n] = u[n]$$

$$\delta[n - k] \longrightarrow h[n - k] = u[n - k]$$

$$a \delta[n - k] \longrightarrow a h[n - k] = a u[n - k]$$

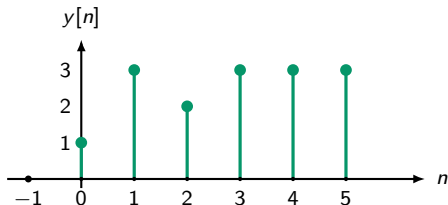
Each sample creates a shifted copy of the impulse response



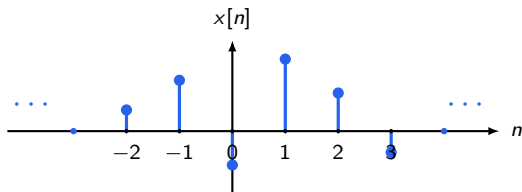
Linearity lets us add the four outputs

Output of the toy echo system

$$y[n] = u[n] + 2u[n-1] - u[n-2] + u[n-3].$$



Any discrete-time input can be built from shifted impulses



each sample comes from one scaled shifted impulse

$$x[n] = \sum_{k=-\infty}^{\infty} x[k] \delta[n - k]$$

The value $x[k]$ becomes the height of the impulse placed at $n = k$.

The two LTI properties again simplify everything!

Time-invariant

If $\delta[n] \longrightarrow h[n]$, then a shifted impulse gives a shifted response:

$$\delta[n - k] \longrightarrow h[n - k].$$

Linear

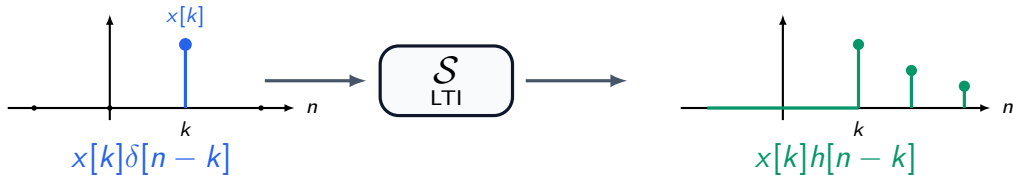
If the shifted impulse is scaled by $x[k]$, the output is scaled by the same amount:

$$x[k]\delta[n - k] \longrightarrow x[k]h[n - k].$$

Each sample creates a shifted copy of the impulse response

Suppose the impulse response of the LTI system is $h[n]$.
Then one input piece $x[k]\delta[n - k]$ produces one output piece $x[k]h[n - k]$.

One input sample at index k



Now add the responses from all samples

Input decomposition becomes output summation

$$x[n] = \sum_k x[k] \delta[n - k]$$

$$\textbf{Therefore, } y[n] = \sum_k x[k] h[n - k]$$

Every term $x[k] \delta[n - k]$ contributes one term $x[k] h[n - k]$ to the output.

Convolution Returns!!!

The LTI input-output formula

$$y[n] = \sum_{k=-\infty}^{\infty} x[k] h[n - k]$$

$$y[n] = (x * h)[n]$$

The bridge thus completes!

1

Find the system's response to the simplest input.

$\delta[n] \rightarrow h[n]$. This $h[n]$ is the impulse response.

2

Break any input signal down into shifted impulses.

$$x[n] = \sum_k x[k] \delta[n - k].$$

3

Shift, scale, and add the impulse responses.

$$y[n] = \sum_k x[k] h[n - k] = (x * h)[n].$$

Please try listening to convolution outputs here!



Let's Listen to Convolutions

Upload or record a sound, choose a system by its impulse response $h[n]$, and hear the convolution $y = x * h$. Turning a formula into something you can listen to.

1 Choose your input signal $x[n]$

Pick a built-in sound, upload a file, or record with your microphone. Short clips (1–3 s) work best.



Click



Beep



Clap



Chirp



Pluck



Upload audio

Record

Loaded: Click

Duration 0.50 s Sample rate 44,100 Hz Samples 22,050 Peak 0.950



Play input

2 Choose the system $h[n]$

The impulse response is the system. Convolution tells you how that system acts on any input.

Exponential Decay Exponential Growth

Thank you!